



Mastering the Node.js Event Loop



What is the Event Loop?

The Event Loop is the core of Node.js's non-blocking architecture. It handles asynchronous callbacks and ensures JavaScript remains single-threaded but non-blocking.

Think of it as a busy manager handling tasks in different queues efficiently switching between them!



Event Loop Phases (in order)

1. Timers – Executes `setTimeout`, `setInterval`.
2. Pending Callbacks – Executes deferred system tasks.
3. Idle, Prepare – Internal use (libuv).
4. Poll – Waits for I/O operations (e.g., `fs.readFile`).
5. Check – Executes `setImmediate` callbacks.
6. Close Callbacks – Handles `on('close')` events.



Microtasks & Priority

Microtasks run **between** the phases of the event loop:

1. `process.nextTick()` – Highest priority.
2. Promise callbacks (`.then`, `queueMicrotask`).
3. Event Loop phases (`setTimeout`, `setImmediate`, etc).

Code Example

```
console.log("start");
setTimeout(() => console.log("timeout"), 0);
setImmediate(() => console.log("immediate"));
fs.readFile("file.txt", () => {
  console.log("file read");
  process.nextTick(() => console.log("tick inside IO"));
  setImmediate(() => console.log("immediate inside IO"));
});
process.nextTick(() => console.log("tick"));
Promise.resolve().then(() => console.log("promise"));
console.log("end");
```

Expected Output

```
start
end
tick
promise
timeout
file read
tick inside IO
immediate
immediate inside IO
```

Key Insights

- ✓ Microtasks (nextTick & Promise) run before moving to the next phase.
- ✓ setImmediate vs setTimeout(..., 0) can behave differently depending on I/O.
- ✓ Poll phase waits if I/O is pending, else skips to Check.

✓ Use this understanding to optimize async code and prevent bugs!